

wCQ bounded: implementazione, validazione e prestazioni in MoStream

Documentazione tecnica per attività di tesi

Progetto MoStream - 22 luglio 2026

Questa relazione studia wCQ, la coda FIFO MPMC wait-free e a memoria bounded proposta da Ruslan Nikolaev e Binoy Ravindran. Il nucleo ufficiale è stato integrato in MoStream tramite un bridge C x86-64 e un wrapper Mojo. Per trasportare valori UInt64 con capacità pubblica esatta, l'adattatore usa due ring wCQ, un array di payload e due contatori padded di admission. La suite di correttezza ha superato test full/empty, wrap, exact-once, bounded fill e topologie fino a 8 producer e 8 consumer, incluse capacità 16, 1024 e 65536. Nella campagna prestazionale di 18 configurazioni e 10 ripetizioni wCQ scala con la contesa e, a 8P-8C con capacità 1024, è 2.48 volte più veloce della CAS originale. Non supera però FAA, Rigtorp o Padded-FAA: raggiunge rispettivamente il 61.1, 57.5 e 49.5 percento del loro throughput in media geometrica. La conclusione riguarda l'adattatore applicativo misurato, non dimostra che il ring wCQ puro sia intrinsecamente più lento. La prova wait-free pubblicata per il nucleo non viene attribuita automaticamente alla composizione bounded e alle operazioni blocking introdotte in MoStream.

Obiettivo dello studio

MoStream usa code bounded per trasferire messaggi fra stage concorrenti. Gli esperimenti precedenti hanno confrontato una MPMC basata su CAS, varianti FAA, Hybrid, Rigtorp, Padded-FAA e un adattamento bounded ispirato a LPRQ. wCQ è stata aggiunta per rispondere a due domande distinte:

1. il nucleo wait-free può essere adattato alla semantica bounded richiesta dal progetto senza perdita o duplicazione dei messaggi?
2. l'uso di FAA e helping consente di superare le code già analizzate quando la contesa cresce?

Le due domande non devono essere confuse. La correttezza del ring upstream non dimostra automaticamente quella dell'adattatore; allo stesso modo, il costo del bridge e dell'indirizzamento non misura isolatamente l'algoritmo pubblicato.

Provenienza e riproducibilità del nucleo

Il sorgente è una copia non modificata di wfring_cas2.h e delle dipendenze lf/ provenienti dal repository ufficiale rusnikola/lfqueue, commit:

708c0052872950dcb15b487fa7a5dd77ce2a2746

Il commit include la correzione del 25 novembre 2023 relativa al valore atteso del CAS nel protocollo slow increment. Le licenze BSD-2-Clause e MIT e gli avvisi di copyright upstream sono conservati nei file vendorizzati. Il file MoStream/lib/wcq_vendor/UPSTREAM.md registra provenienza e checksum SHA-256.

La pubblicazione di riferimento è wCQ: A Fast Wait-Free Queue with Bounded Memory Usage, SPAA 2022. La versione consultabile è disponibile su arXiv con identificatore 2201.02179.

Proprietà dichiarate dal lavoro originale

wCQ è una FIFO Multiple Producer, Multiple Consumer a capacità fissa. Il lavoro la presenta come wait-free: ogni operazione del ring completa in un numero finito di passi sotto le assunzioni del modello. La memoria è bounded: non vengono concatenati segmenti allocati dinamicamente per superare un ring in stallo.

Le assunzioni rilevanti sono:

- capacità logica n potenza di due;
- numero k di thread partecipanti non superiore a n ;
- Fetch-And-Add e atomic OR implementati dall'hardware;
- CAS a doppia parola, CAS2, per coppie contatore-descrittore ed entry-note;
- record per-thread collegati in una lista usata dal protocollo di helping;
- indici interni nell'intervallo previsto dal ring, non payload arbitrari.

Su x86-64 il CAS2 corrisponde a CMPXCHG16B e richiede allineamento a 16 byte. Il bridge controlla a runtime la presenza della feature CPU CX16. Il port corrente non è offerto su architetture diverse da x86-64.

Da SCQ a wCQ

wCQ deriva da SCQ, una coda lock-free basata su ring. Head e Tail sono avanzati principalmente con FAA, evitando il ciclo CAS sul contatore globale tipico di molte MPMC. Ogni posizione logica è associata a una entry fisica e a un ciclo, così lo stesso spazio può essere riutilizzato senza confondere generazioni.

Per una capacità logica n il ring mantiene $2n$ entry fisiche, ma contiene al massimo n indici. Il raddoppio offre margine fra producer e consumer che raggiungono posizioni in momenti diversi. Un threshold limita le scansioni a vuoto e contribuisce al progresso del ring.

SCQ è lock-free: almeno un thread progredisce, ma un singolo thread può essere ritardato indefinitamente. wCQ aggiunge un percorso lento cooperativo per rafforzare la garanzia a wait-free.

Fast path e slow path

Il fast path conserva l'idea SCQ. Enqueue e dequeue prenotano posizioni con FAA e tentano di aggiornare la entry corrispondente. Nel sorgente vendorizzato la patience è limitata a 16 tentativi per enqueue e 64 per dequeue.

Quando il fast path non completa entro il limite, l'operazione pubblica una richiesta nel proprio record per-thread. La richiesta contiene sequenze per leggere uno snapshot coerente, head o tail locale, posizione iniziale e tipo di operazione. Gli altri thread controllano periodicamente i record; il delay upstream è 16 operazioni.

Il protocollo slow increment coordina gli helper affinché convergano sulla stessa posizione globale. La coppia atomica contiene il contatore e il riferimento alla fase di helping. Entry e note usano anch'esse aggiornamenti a doppia parola. Il bit FIN termina gli helper quando il risultato è stato prodotto; un bit Enq impedisce al consumer di osservare un inserimento non ancora finalizzato.

Il punto essenziale non è che il percorso lento sia più veloce. Esso limita il numero di tentativi individuali e permette agli altri thread di completare l'operazione del thread in difficoltà.

Perché è stato usato un bridge C nativo

Una prima trascrizione diretta in Mojo ha superato i test sequenziali e parte dei test concorrenti, ma ha mostrato hang e perdita intermittente a elevata contesa quando veniva attivato il percorso lento. L'algoritmo upstream accede intenzionalmente alla metà alta di oggetti atomici a 128 bit con operazioni a 64 bit, combinandole con CMPXCHG16B sulla coppia completa.

L'API atomica Mojo usata nel progetto non fornisce una garanzia esplicita per accessi atomici sovrapposti di larghezza diversa. Conservare quel port avrebbe reso impossibile distinguere un difetto algoritmico da una traduzione non fedele al memory model.

La versione finale richiama quindi il sorgente C ufficiale tramite FFI. Questa scelta privilegia la fedeltà e la correttezza sperimentale, ma introduce un costo per chiamata e rende il confronto non completamente indipendente dal linguaggio di implementazione.

Adattamento di indici a payload UInt64

Il ring wCQ trasporta indici bounded. Per esporre una coda di valori si applica lo schema di indizione descritto nel paper:

aq	ring inizialmente vuoto degli indici pubblicati
fq	ring inizialmente pieno degli indici liberi
values[n]	array fisso dei payload UInt64
item_count	indici già visibili in aq
free_count	indici già visibili in fq

Enqueue preleva un indice da fq, scrive values[index] e pubblica l'indice in aq. Dequeue preleva da aq, legge values[index] e restituisce l'indice a fq. Poiché circolano esattamente n indici, nessuno dei due ring può contenere più della capacità logica. La memoria non cresce dopo la costruzione.

L'API corrente conserva ogni bit di UInt64, incluso UINT64_MAX. Non è ancora un adattatore generico per un T Mojo arbitrario; payload più grandi richiederebbero una tabella Mojo o un ABI esplicito.

Problema osservato con il retry a vuoto

La prima composizione nativa usava dequeue con controllo empty e ripeteva immediatamente l'operazione quando aq o fq risultavano vuoti. Ogni risposta empty è ammessa dal ring, ma una successione aggressiva di tentativi può far avanzare Head molto più rapidamente delle nuove pubblicazioni.

Nei test applicativi questo comportamento ha prodotto non terminazione intermittente: i consumer attendevano messaggi già prodotti o i producer attendevano indici che non riuscivano più a intercettare. La prova senza sentinel, basata su bitmap exact-once e contatore consumed, ha riprodotto lo stesso problema; il sentinel non era quindi la causa.

Questa osservazione non dimostra un errore del ring wCQ. Mostra un problema di composizione fra operazioni singole che possono restituire empty e un wrapper blocking che le richiama in un ciclo caldo.

Contatori di admission

La soluzione finale evita di interrogare un ring quando non esiste un token pubblicato. item_count parte da zero e free_count da n; ciascuno è isolato su

una cache line da 128 byte, coerente con LF_CACHE_BYTES dell'artifact.

try_enqueue esegue:

1. load di free_count; se zero ritorna WOULD_BLOCK
2. un solo strong CAS free_count da c a c-1
3. fq.dequeue(nonempty=true)
4. values[index] = value
5. aq.enqueue(index, nonempty=false)
6. item_count.fetch_add(1, RELEASE)

try_dequeue è simmetrica:

1. load di item_count; se zero ritorna WOULD_BLOCK
2. un solo strong CAS item_count da c a c-1
3. aq.dequeue(nonempty=true)
4. value = values[index]
5. fq.enqueue(index, nonempty=true)
6. free_count.fetch_add(1, RELEASE)

Un CAS perso restituisce WOULD_BLOCK anche se, astrattamente, esiste ancora spazio o un elemento. Il fallimento è quindi debole e transitorio. push e pop richiamano i tentativi con pause e yield periodico.

I contatori eliminano il runaway dell'Head osservato nei test, ma reintroducono un hot spot CAS sul percorso applicativo. Questo costo è una delle spiegazioni plausibili delle prestazioni inferiori a Padded-FAA; i dati non isolano però causalmente CAS, FFI, doppio ring e indirezione.

API esposta a Mojo

Il wrapper è definito in MoStream/WCQ_queue.mojo:

```
from MoStream.WCQ_queue import WCQQueue

var queue = WCQQueue(1024, max_threads=8)
queue.push(thread_id=0, item=UInt64(42))
var value = queue.pop(thread_id=1)
```

Metodi disponibili:

- try_push(thread_id, item): None al successo, Some(item) su WOULD_BLOCK;
- try_pop(thread_id): Some(value) al successo, None su WOULD_BLOCK;
- push(thread_id, item): inserimento blocking con spin/retry;
- pop(thread_id): estrazione blocking con spin/retry.

La capacità deve essere una potenza di due almeno pari a otto. Ogni chiamante concorrente deve usare un thread_id distinto e denso nell'intervallo [0,max_threads). Lo stesso id può essere riutilizzato dopo la conclusione della chiamata precedente. La coda può essere mossa o distrutta soltanto dopo la quiescenza di tutti i worker.

Boundedness e footprint

La capacità pubblica è esatta: al massimo n payload sono accettati prima che try_push restituisca l'elemento. Il bridge alloca due ring, ognuno con 2n entry fisiche da 128 bit, un array di n UInt64 e due array di max_threads record per-thread. I record upstream occupano 128 byte su x86-64.

Trascurando header e arrotondamenti, i soli array dei ring richiedono circa 64n byte complessivi, perché ciascun ring contiene 2n coppie da 16 byte. I payload aggiungono 8n byte; i descrittori aggiungono circa 256 x max_threads byte per i due ring.

tab(), box;

Capacità n Ring + payload, stima Descrittori con 16 thread

16 circa 1.1 KiB circa 4 KiB
1024 circa 72 KiB circa 4 KiB
65536 circa 4.5 MiB circa 4 KiB

La stima non include strutture, allineamento, allocator metadata e contatori. Rispetto a una FAA compatta, il footprint è maggiore; rispetto a strutture unbounded non cresce durante l'esecuzione.

Garanzie di progresso: distinzione necessaria

Il ring upstream viene presentato e dimostrato wait-free sotto le assunzioni del paper. La formulazione corretta per il codice MoStream è più limitata.

Ogni try dell'adattatore esegue un solo CAS di admission e poi un'operazione wCQ. Non contiene un loop CAS applicativo, ma la composizione a due ring e contatori non è accompagnata in questo progetto da una prova formale separata di linearizzabilità o wait-freedom.

Un thread sospeso dopo aver decrementato un contatore possiede temporaneamente un token. Altri try possono osservare WOULD_BLOCK; push e pop possono attendere indefinitamente se la coda resta piena o vuota, se il proprietario non riparte o se continuano a perdere l'admission. Le operazioni blocking non sono quindi wait-free.

Gli stress test aumentano la fiducia nell'implementazione, ma non sostituiscono una dimostrazione rispetto a tutti gli interleaving e al memory model.

Validazione funzionale

La suite Tests/test_wcq_native.c verifica:

- argomenti invalidi, capacità e metadata;
- full ed empty esatti e preservazione dell'output su empty;
- 25000 wrap completi a capacità otto e 128 a capacità 1024;
- supporto di tutti i bit UInt64, incluso UINT64_MAX;
- bounded fill concorrente con 1, 2, 4 e 8 producer;
- exact-once tramite bitmap atomica, senza affidarsi al solo checksum;
- ordine crescente per producer osservato da ciascun consumer;
- matrici 1P-1C, 2P-2C, 4P-4C e 8P-8C;
- workload 8P-1C e 1P-8C;
- capacità 16, 1024 e 65536 e drain finale vuoto;
- watchdog per distinguere un hang da un test semplicemente lento.

Cinque suite in processi isolati sono terminate senza failure o timeout. Le build ASan e UBSan non hanno segnalato errori; anche TSan non ha prodotto race report. L'assembly inline CAS2 vendorizzato non è necessariamente osservabile integralmente da TSan, quindi questo esito non equivale a una prova.

I tre test Mojo verificano inoltre bounded FIFO sequenziale, exact-once blocking concorrente e API try concorrente fino a 8P-8C.

Metodologia prestazionale

Il benchmark comune costruisce la coda prima del timer. Ogni producer inserisce un intervallo disgiunto di interi; l'ultimo producer inserisce un sentinel per consumer. I consumer mantengono count e checksum locali, evitando un RMW condiviso per messaggio. Dopo il join ogni campione è accettato soltanto se:

```
count      = numero totale atteso
checksum = N x (N - 1) / 2
```

Parametri della campagna results_wcq.csv:

- 100000 messaggi per producer;
- 10 ripetizioni per configurazione con ordine delle code ruotato;
- topologie 1P-1C, 2P-2C, 4P-4C, 8P-8C, 1P-8C e 8P-1C;
- capacità 16, 1024 e 65536;
- media, deviazione standard e intervallo di confidenza 95 percento Student-t;
- rapporti wCQ/Padded-FAA e wCQ/Rigtorp paired per ripetizione.

Tutti i 180 campioni di configurazione sono risultati validi. L'ambiente è un Intel Xeon Gold 5512U, 28 core e 56 thread hardware, un socket e un nodo NUMA, Linux 5.15, GCC 11.4.0, Mojo 1.0.0b1 e ottimizzazione O3.

Non sono stati applicati pinning esplicito, isolamento dei core o controllo della frequenza. I test più brevi, in particolare 1P-1C capacità 65536, hanno intervalli ampi e non devono sostenere da soli una conclusione causale.

Risultati completi di throughput

Valori in milioni di messaggi al secondo. Best considera tutte le dieci code del runner; H1 indica Hybrid con soglia uno.

```
tab( ), box;
Topologia Capacità CAS FAA Rigtorp Padded wCQ Best
1P-1C 16 3.854 5.389 3.831 4.494 2.146 FAA
1P-1C 1024 4.973 6.726 3.413 4.856 2.181 FAA
1P-1C 65536 12.407 13.955 9.884 20.293 4.048 Padded
2P-2C 16 3.503 5.438 5.708 6.283 2.795 Padded
2P-2C 1024 3.609 5.677 4.851 5.556 2.852 FAA
2P-2C 65536 3.363 5.569 5.735 6.175 2.672 Padded
4P-4C 16 2.460 5.874 6.570 7.346 4.199 Padded
4P-4C 1024 3.298 7.895 8.340 9.420 5.377 Padded
4P-4C 65536 3.420 8.616 8.459 9.823 5.757 Padded
8P-8C 16 2.386 8.887 9.320 9.797 5.956 Padded
8P-8C 1024 2.665 9.709 8.693 11.130 6.598 Padded
8P-8C 65536 2.870 9.072 8.760 10.179 6.920 Padded
1P-8C 16 3.479 4.691 8.071 8.036 3.933 Rigtorp
1P-8C 1024 3.406 4.283 7.293 7.451 3.891 Padded
1P-8C 65536 3.781 4.362 6.785 6.507 3.316 Rigtorp
8P-1C 16 3.234 4.932 7.346 8.094 3.898 Padded
8P-1C 1024 4.095 5.343 7.389 8.222 4.239 Padded
8P-1C 65536 4.305 5.855 7.511 8.681 4.393 H1
```

Scalabilità bilanciata a capacità 1024

La tabella successiva riporta media wCQ, IC 95 percento e rapporti paired.

```
tab( ), box;
Topologia wCQ Mmsg/s IC 95% wCQ/Padded wCQ/Rigtorp
1P-1C 2.181 +/-0.059 0.450x 0.643x
2P-2C 2.852 +/-0.530 0.539x 0.638x
4P-4C 5.377 +/-0.107 0.572x 0.645x
8P-8C 6.598 +/-0.101 0.593x 0.759x
```

wCQ cresce da 2.181 a 6.598 Mmsg/s passando da 1P-1C a 8P-8C. Nello stesso scenario a 8P-8C, CAS scende a 2.665 Mmsg/s: wCQ è quindi 2.48 volte più veloce. Il risultato è coerente con l'obiettivo di ridurre i retry CAS sul nucleo conteso.

Il miglioramento di scalabilità non basta a vincere. Padded-FAA raggiunge 11.130 Mmsg/s ed è circa 1.69 volte più veloce; Rigtorp raggiunge 8.693 ed è circa 1.32 volte più veloce. Gli IC 95 percento dei rapporti paired restano interamente sotto uno.

Risultato aggregato

Sulle 18 combinazioni topologia-capacità:

- Padded-FAA è prima in 12 casi e ha media geometrica 7.953 Mmsg/s;
- FAA vince tre casi, Rigtorp due e Hybrid-1 uno;
- wCQ non vince alcun caso e ha media geometrica 3.941 Mmsg/s;
- il rapporto geometrico wCQ/Padded-FAA è 0.495x;
- il rapporto geometrico wCQ/Rigtorp è 0.575x;
- il rapporto geometrico wCQ/FAA è 0.611x;
- wCQ supera CAS in 11 casi su 18 e bounded-LPRQ in 16 su 18.

La media geometrica assegna uguale peso relativo alle configurazioni e riduce l'influenza dei valori assoluti più elevati. Non sostituisce l'analisi per topologia, ma fornisce una sintesi più corretta della sola media aritmetica.

Interpretazione per carico

Bassa contesa

Con 1P-1C i due ring, il payload indiretto, l'admission e la FFI sono costi non ammortizzati. A capacità 1024 wCQ raggiunge il 45.0 per cento di Padded-FAA e il 64.3 per cento di Rigtorp. Una SPSC dedicata resta la soluzione corretta quando esiste un solo producer e un solo consumer.

Contesa bilanciata elevata

Da 4P-4C wCQ supera CAS e bounded-LPRQ e riduce il divario da Rigtorp. Il rapporto wCQ/Rigtorp cresce fino a 0.759 a 8P-8C capacità 1024 e 0.790 a capacità 65536. Il nucleo FAA e il minor numero di retry rispetto a CAS contribuiscono plausibilmente, ma il benchmark non isola le singole cause.

Workload sbilanciati

In 1P-8C e 8P-1C wCQ non supera Rigtorp o Padded-FAA. L'admission globale è contesa da molti thread sul lato multiplo, mentre una coda SPMC o MPSC specializzata potrebbe rimuovere l'RMW dal lato a singolo proprietario.

Capacità

Il comportamento di wCQ varia meno di alcune code padded, ma il footprint di due ring cresce linearmente. I risultati 1P-1C capacità 65536 sono rumorosi per tutte le code e non autorizzano conclusioni sulla sola cache locality.

Confronto qualitativo

```
tab( ),box;
Proprietà CAS-MPMC FAA compatta Rigtorp wCQ adapter
Prenotazione hot CAS loop FAA FAA FAA ring + CAS admission
Slot padded No No Si entry 128-bit remapped
Payload diretto diretto diretto indiretto UInt64
Memoria bounded bounded bounded bounded, due ring
Progresso blocking lock-free core ticket blocking ticket blocking spin/retry
Helping No No No Si, nel ring upstream
CAS2 richiesto No No No Si
Architettura port x86-64 x86-64 x86-64 x86-64 CX16
```

La riga sul progresso descrive l'API applicativa misurata. Non deve essere letta come negazione della proprietà wait-free del ring wCQ pubblicato.

Applicabilità a MoStream

Per il percorso dati corrente, Padded-FAA resta la scelta generale orientata al throughput. Rigtorp è una baseline robusta e particolarmente competitiva in alcuni carichi sbilanciati. wCQ è applicabile e corretta nei test, ma non è una sostituzione prestazionale conveniente nella forma attuale.

wCQ può diventare interessante quando:

1. la garanzia di progresso del nucleo è più importante del throughput medio;
2. l'applicazione trasporta direttamente indici unici e può evitare values e fq;
3. la topologia garantisce quando un ring è nonempty senza un contatore CAS;
4. si implementa una admission sharded o basata su FAA con semantica dimostrata;
5. si vuole studiare l'effetto di preemption e helping rispetto ai ticket

irrevocabili di FAA e Rigtorp.

Una selezione runtime per messaggio aggiungerebbe un ulteriore branch caldo. L'eventuale adozione dovrebbe scegliere il tipo concreto alla costruzione del Communicator.

Limiti dello studio

- Una sola macchina, un solo sistema operativo e una versione beta di Mojo.
- Nessun pinning o isolamento dei core nella campagna principale.
- Solo throughput medio; non sono misurate p50, p95, p99, fairness o energia.
- Payload UInt64, non oggetti Mojo di dimensione variabile.
- Confronto fra code Mojo e un nucleo C chiamato via FFI.
- I sanitizer non osservano necessariamente ogni effetto dell'assembly CAS2.
- Nessuna prova formale della composizione due-ring più admission.
- Un CAS perso da try produce un fallimento debole anche quando la coda potrebbe astrattamente avere spazio o dati.
- Le operazioni blocking possono consumare CPU durante attese prolungate.

Lavoro futuro

1. Misurare separatamente ring wCQ diretto e adattatore a due ring.
2. Creare benchmark nativi equivalenti per tutte le code, eliminando il fattore linguaggio e FFI.
3. Provare admission per-thread o sharded e confrontarla col CAS globale.
4. Integrare payload Mojo generici con storage preallocato e ownership verificata.
5. Eseguire model checking o una prova della composizione e del memory ordering.
6. Sospendere deliberatamente un thread nel fast path, slow path e dopo admission.
7. Misurare latenza massima e distribuzione per verificare il beneficio del helping, non soltanto il throughput.
8. Ripetere con core isolati, pinning, almeno 30 campioni e frequenza controllata.
9. Usare perf per cache miss, invalidazioni, CAS falliti, context switch e TLB.
10. Valutare pipeline MoStream end-to-end con topologie SPSC, SPMC, MPSC e MPMC.

Conclusione

L'esperimento dimostra che è possibile integrare il nucleo wCQ ufficiale in una coda bounded MoStream che supera una validazione concorrente ampia. La prima composizione a retry vuoto non era adatta a un wrapper blocking; i contatori di admission hanno eliminato la non terminazione osservata e reso la capacità pubblica esatta.

wCQ scala meglio della coda CAS sotto contesa: a 8P-8C capacità 1024 raggiunge 6.598 Mmsg/s contro 2.665. Tuttavia l'adattatore non supera FAA, Rigtorp o

Padded-FAA in nessuna delle 18 configurazioni. Padded-FAA conserva quindi il ruolo di default orientato al throughput.

La formulazione corretta per la tesi è: wCQ è una soluzione promettente per il progresso wait-free del ring e per scenari a indici diretti; nell'adattatore bounded UInt64 di MoStream, doppio ring, FFI e CAS di admission riducono il vantaggio prestazionale. Correttezza sperimentale e proprietà formali della composizione devono rimanere esplicitamente distinte.

Artefatti riproducibili

- MoStream/WCQ_queue.mojo: wrapper Mojo;
- MoStream/lib/wcq_native.c e wcq_native.h: bridge bounded UInt64;
- MoStream/lib/wcq_vendor: sorgente upstream, licenze e provenienza;
- Tests/test_wcq_native.c: suite nativa estesa;
- Tests/test_wcq_queue.mojo, test_wcq_concurrent.mojo e test_wcq_try_concurrent.mojo: test Mojo;
- Benchmarks/QueueBenchmark/queue_benchmark.mojo: benchmark;
- Benchmarks/QueueBenchmark/run_suite.py: runner statistico;
- Benchmarks/QueueBenchmark/results_wcq.csv: risultati completi;
- Benchmarks/QueueBenchmark/analyze_wcq_results.py: analisi aggregata.

Riferimenti

1. R. Nikolaev e B. Ravindran, wCQ: A Fast Wait-Free Queue with Bounded Memory Usage, SPAA 2022, pp. 307-319, DOI <https://doi.org/10.1145/3490148.3538572>, preprint <https://arxiv.org/abs/2201.02179>.
2. R. Nikolaev, lqueue repository e implementazione wfring_cas2.h, <https://github.com/rusnikola/lqueue>, commit 708c005.
3. R. Nikolaev, A Scalable, Portable, and Memory-Efficient Lock-Free FIFO Queue, DISC 2019, <https://rusnikola.github.io/files/ringpaper-disc.pdf>.
4. MoStream, risultati locali Benchmarks/QueueBenchmark/results_wcq.csv, campagna del 22 luglio 2026.